

Unit 2

Data Encapsulation and Inheritance

2.1 Access Modifiers in C++

Access Modifiers or Access Specifiers in a [class](#) are used to assign the accessibility to the class members. That is, it sets some restrictions on the class members not to get directly accessed by the outside functions. There are 3 types of access modifiers available in C++:

1. **Public**
2. **Private**
3. **Protected**

Note: If we do not specify any access modifiers for the members inside the class then by default the access modifier for the members will be **Private**.

Let us now look at each one these access modifiers in details:

1. Public: All the class members declared under the public specifier will be available to everyone. The data members and member functions declared as public can be accessed by other classes and functions too. The public members of a class can be accessed from anywhere in the program using the direct member access operator (.) with the object of that class.

2. Private: The class members declared as *private* can be accessed only by the member functions inside the class. They are not allowed to be accessed directly by any object or function outside the class. Only the member functions or the [friend functions](#) are allowed to access the private data members of a class.

3. Protected: Protected access modifier is similar to private access modifier in the sense that it can't be accessed outside of it's class unless with the help of friend class, the difference is that the class members declared as Protected can be accessed by any subclass(derived class) of that class as well.

C++ Classes and Objects

Class: A class in C++ is the building block, that leads to Object-Oriented programming. It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class. A C++ class is like a blueprint for an object. For Example: Consider the Class of **Cars**. There may be many cars with different names and brand but all of them will share some common properties like all of them will have *4 wheels*, *Speed Limit*, *Mileage range* etc. So here, Car is the class and wheels, speed limits, mileage are their properties.

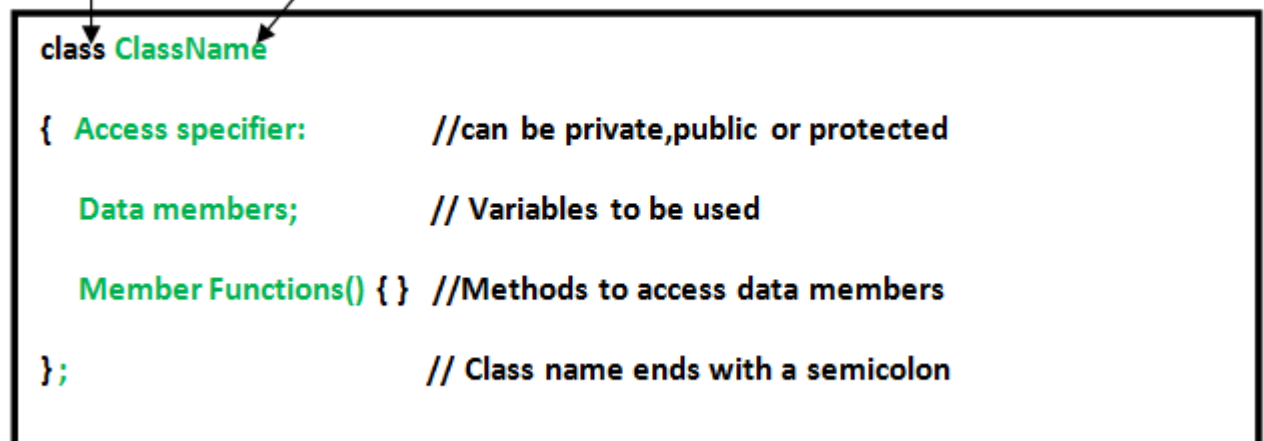
- A Class is a user defined data-type which has data members and member functions.
- Data members are the data variables and member functions are the functions used to manipulate these variables and together these data members and member functions defines the properties and behavior of the objects in a Class.
- In the above example of class *Car*, the data member will be *speed limit, mileage* etc and member functions can be *apply brakes, increase speed* etc.

An **Object** is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

Defining Class and Declaring Objects

A class is defined in C++ using keyword `class` followed by the name of class. The body of class is defined inside the curly brackets and terminated by a semicolon at the end.

keyword user-defined name



Declaring Objects: When a class is defined, only the specification for the object is defined; no memory or storage is allocated. To use the data and access functions defined in the class, you need to create objects.

Syntax:

`ClassName ObjectName;`

Accessing data members and member functions: The data members and member functions of class can be accessed using the dot('.') operator with the object. For example if the name of object is *obj* and you want to access the member function with the name *printName()* then you will have to write *obj.printName()* .

Accessing Data Members

The public data members are also accessed in the same way given however the private data members are not allowed to be accessed directly by the object. Accessing a data member depends solely on the access control of that data

member.

This access control is given by [Access modifiers in C++](#). There are three access modifiers : **public**, **private** and **protected**.

Member Functions in Classes

There are 2 ways to define a member function:

- Inside class definition
- Outside class definition

To define a member function outside the class definition we have to use the scope resolution :: operator along with class name and function name.

Concept and use of enum

Enumeration is a user defined datatype in C/C++ language. It is used to assign names to the integral constants which makes a program easy to read and maintain. The keyword “enum” is used to declare an enumeration.

The following is the syntax of enums.

```
enum enum_name{const1, const2, ..... };
```

Here,

enum_name – Any name given by user.

const1, const2 – These are values of type flag.

The enum keyword is also used to define the variables of enum type. There are two ways to define the variables of enum type as follows –

```
enum colors{red, black};  
enum suit{heart, diamond=8, spade=3, club};
```

The following is an example of enums.

Example

```
#include <iostream>  
  
using namespace std;  
  
enum colors{red=5, black};  
enum suit{heart, diamond=8, spade=3, club};  
  
int main() {  
    cout <<"The value of enum color : "<<red<<","<<black;  
    cout <<"\n\nThe default value of enum suit :  
"<<heart<<","<<diamond<<","<<spade<<","<<club;
```

```
return 0;  
  
}
```

Output

```
The value of enum color : 5,6  
The default value of enum suit : 0,8,3,4
```

2.4 Concept of Data Hiding ,abstraction and encapsulation with example:

In normal terms, **Encapsulation** is defined as wrapping up of data and information under a single unit. In Object-Oriented Programming, Encapsulation is defined as binding together the data and the functions that manipulate them.

Consider a real-life example of encapsulation, in a company, there are different sections like the accounts section, finance section, sales section etc. The finance section handles all the financial transactions and keeps records of all the data related to finance. Similarly, the sales section handles all the sales-related activities and keeps records of all the sales. Now there may arise a situation when for some reason an official from the finance section needs all the data about sales in a particular month. In this case, he is not allowed to directly access the data of the sales section. He will first have to contact some other officer in the sales section and then request him to give the particular data. This is what encapsulation is. Here the data of the sales section and the employees that can manipulate them are wrapped under a single name “sales section”.

Abstraction: Data abstraction is one of the most essential and important features of object-oriented programming in C++. Abstraction means displaying only essential information and hiding the details. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.

Consider a real-life example of a man driving a car. The man only knows that pressing the accelerators will increase the speed of the car or applying brakes will stop the car but he does not know about how on pressing accelerator the speed is actually increasing, he does not know about the inner mechanism of the car or the implementation of accelerator, brakes etc in the car. This is what abstraction is.

- *Abstraction using Classes*: We can implement Abstraction in C++ using classes. The class helps us to group data members and member functions using available access specifiers. A Class can decide which data member will be visible to the outside world and which is not.
- *Abstraction in Header files*: One more type of abstraction in C++ can be header files. For example, consider the pow() method present in math.h header file. Whenever we need to calculate the power of a number, we

simply call the function `pow()` present in the `math.h` header file and pass the numbers as arguments without knowing the underlying algorithm according to which the function is actually calculating the power of numbers.

2.5 Concept of Inheritance and types of Inheritance

Inheritance is the capability of one class to acquire properties and characteristics from another class. The class whose properties are inherited by other class is called the **Parent** or **Base** or **Super** class. And, the class which inherits properties of other class is called **Child** or **Derived** or **Sub** class.

Inheritance makes the code reusable. When we inherit an existing class, all its methods and fields become available in the new class, hence code is reused.

NOTE: All members of a class except Private, are inherited

Basic Syntax of Inheritance

```
class Derivedclassname : access_mode Baseclassname
```

Example:

Class base

```
{ int a
Public:
    Void geta( ) { a=10; }
    Void puta( ) { cout<<a ;}
};
```

Class derived : public base

```
{ int d
Public:
    Void getd( ) { d=10; }
    Void putd( ) { cout<<d; }
};
```

```
Int main()
{
derived d1;
d1.geta();
d1.puta();
d1.getd();
d1.putd();
getch();
return(0);
}
```

Access Modifiers and Inheritance: Visibility of Class Members

Depending on Access modifier used while inheritance, the availability of class members of Super class in the sub class changes. It can either be private, protected or public.

1) Public Inheritance

This is the most used inheritance mode. In this the protected member of super class becomes protected members of sub class and public becomes public.

```
class Derivedclassname : public Baseclassname
```

2) Private Inheritance

In private mode, the protected and public members of super class become private members of derived class.

```
class Derivedclassname : Baseclassname // By default its private inheritance
```

3) Protected Inheritance

In protected mode, the public and protected members of Super class becomes protected members of Sub class.

```
class Derivedclassname : protected Baseclassname
```

Table showing all the Visibility Modes

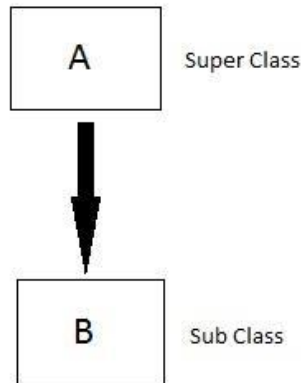
	Derived Class	Derived Class	Derived Class
Base class	Public Mode	Private Mode	Protected Mode
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Protected	Private	Protected
Public	Public	Private	Protected

Types of Inheritance in C++

1. Single Inheritance
2. Multiple Inheritance
3. Hierarchical Inheritance
4. Multilevel Inheritance
5. Hybrid Inheritance (also known as Virtual Inheritance)

Single Inheritance in C++

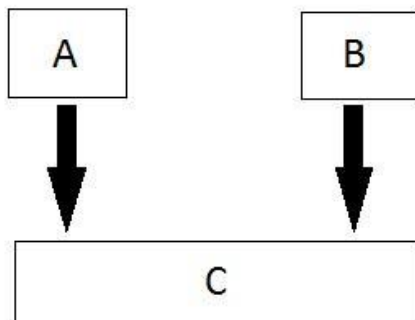
In this type of inheritance one derived class inherits from only one base class. It is the most simplest form of Inheritance.



```
Class A
{ public : void displayA( ) { cout<<"base" ; }
};
Class B : public A
{ public : void displayB( ) { cout<<"derived" ; }
};
int main()
{ B b1;
  b1.displayA();
  b1.displayB();
  return(0);
}
```

Multiple Inheritance in C++

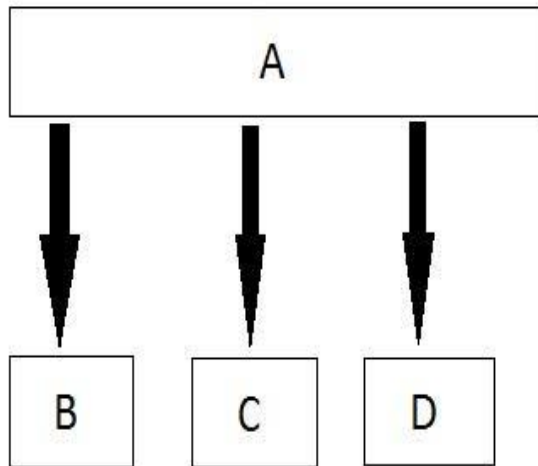
In this type of inheritance a single derived class may inherit from two or more than two base classes.



```
Class A
{ public : void displayA( ) { cout<<"classA" ; }
};
Class B
{ public : void displayB( ) { cout<<"classB" ; }
};
Class C : public A , public B
{ public : void displayC( ) { cout<<"classC" ; }
};
int main()
{ C c1;
  c1.displayA();
  c1.displayB();
  c1.displayC();
  return(0);
}
```

Hierarchical Inheritance in C++

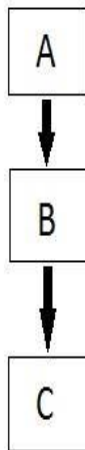
In this type of inheritance, multiple derived classes inherits from a single base class.



```
Class A
{ public : void displayA( ) { cout<<"classA" ; }
};
Class B : public A
{ public : void displayB( ) { cout<<"classB" ; }
};
Class C : : public A
{ public : void displayC( ) { cout<<"classC" ; }
};
Class D: public A
{ public : void displayD( ) { cout<<"classD" ; }
};
int main()
{ B b1;
  C c1;
  D d1;
  b1.displayA();
  c1.displayA();
  d1.displayA();
  b1.displayB( ) ;
  c1.displayC( ) ;
  d1.displayD( ) ;
  return (0);
}
```

Multilevel Inheritance in C++

In this type of inheritance the derived class inherits from a class, which in turn inherits from some other class. The Super class for one, is sub class for the other.

**Class A**

```
{ public : void displayA( ) { cout<<"classA" ; }  
};
```

Class B : public A

```
{ public : void displayB( ) { cout<<"classB" ; }  
};
```

Class C : public B

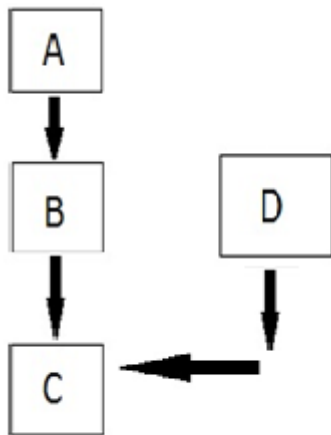
```
{ public : void displayC( ) { cout<<"classC" ; }  
};
```

int main()

```
{ C c1;  
  c1.displayA();  
  c1.displayB();  
  c1.displayC();  
  return(0);  
}
```

Hybrid (Virtual) Inheritance in C++

Hybrid Inheritance is combination of Hierarchical and Multilevel Inheritance.



Class A

```
{ public : void displayA( ) { cout<<"classA" ; }  
};
```

Class D:

```
{ public : void displayD( ) { cout<<"classD" ; }  
};
```

Class B : public A

```
{ public : void displayB( ) { cout<<"classB" ; }  
};
```

Class C : public B, public D

```
{ public : void displayC( ) { cout<<"classC" ; }  
};
```

int main()

```
{ C c1;  
  c1.displayA();  
  c1.displayB();  
  c1.displayD();  
  c1.displayC();  
  return(0);  
}
```

Order of Constructor Call with Inheritance in C++

Base class constructors are always called in the derived class constructors. Whenever you create derived class object, first the base class default constructor is executed and then the derived class's constructor finishes execution.

Points to Remember

1. Whether derived class's default constructor is called or parameterised is called, base class's default constructor is always called inside them.
2. To call base class's parameterised constructor inside derived class's parameterised constructor, we must mention it explicitly while declaring derived class's parameterized constructor.

Base class Default Constructor in Derived class Constructors

```
class Base
{
    int x;
public:
    // default constructor
    Base()
    {
        x=10;
        cout << "Base default constructor\n";
    }
};
class Derived : public Base
{
    int y;
public:
    // default constructor
    Derived()
    {
        cout << "Derived default constructor\n";
    }
    // parameterized constructor
    Derived(int i)
    {
        y=i;
        cout << "Derived parameterized constructor\n";
    }
};
int main()
{
    Derived d1;
    Derived d2(10);
}
```

Output

```
Base default constructor
Derived default constructor
Base default constructor
Derived parameterized constructor
```

```
#include<iostream.h>
#include<conio.h>
```

//parameterised constructor in single inheritance

```
class A
{ int a;
public :
    A(int x) { a=x; }
    void putA() { cout<<a; }

};
class B: public A
{ int b;
public :
    B(int x,int y):A(y)
    {
        b=x;
    }
    void putB() { cout<<b; }
};
int main()
{ clrscr();
  B b1(2,3);
  b1.putA();
  b1.putB();
return(0);
}
```

//parameterised constructor in multiple inheritance

```
class A
{ int a;
public :
    A(int x)
    { a=x; }
    void putA() { cout<<a; }

};
class B
{ int b;
public :
    B(int x)
    {
        b=x;
    }
};
```

```

    }
    void putB() { cout<<b; }
};
class C : public A ,public B
{ int c;
public :
    C(int x,int y,int z) : A(x),B(y)
    {
        c=z;
    }
    void putC() { cout<<c; }
};
int main()
{ clrscr();
  C c1(1,2,3);
  c1.putA();
  c1.putB();
  c1.putC();
return(0);
}

```

//parameterized constructor in multilevel inheritance

```

class A
{ int a;
public :
    A(int x)
    { a=x; }
    void putA() { cout<<a; }
};
class B : public A
{ int b;
public :
    B(int x,int y):A(x)
    {
        b=y;
    }
    void putB() { cout<<b; }
};
class C : public B
{ int c;
public :
    C(int x,int y,int z):B(x,y)
    {
        c=z;
    }
};

```

```

    }
    void putC() { cout<<c; }
};
int main()
{   clrscr();
    C c1(1,2,3);
    c1.putA();
    c1.putB();
    c1.putC();
return(0);
}

```

2.6 Constructors and Destructors:

Constructors in C++

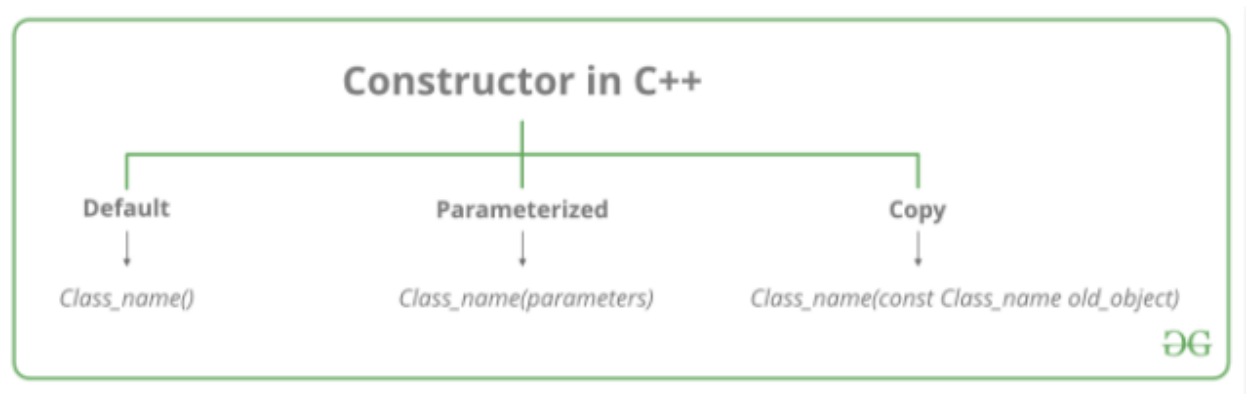
What is constructor?

A constructor is a member function of a class which initializes objects of a class. In C++, Constructor is automatically called when object(instance of class) create. It is special member function of the class.

How constructors are different from a normal member function?

A constructor is different from normal functions in following ways:

- Constructor has same name as the class itself
- Constructors don't have return type
- A constructor is automatically called when an object is created.
- If we do not specify a constructor, C++ compiler generates a default constructor for us (expects no parameters and has an empty body).



Types of Constructors

Default Constructors: Default constructor is the constructor which doesn't take any argument. It has no parameters.

```
class construct
{
    int a, b;
public:
    // Default Constructor
    construct()
    {
        a = 10;
        b = 20;
    }
    void putdata() { cout<<a<<b; }
};

int main()
{
    // Default constructor called automatically
    // when the object is created
    construct c;
    cout << "value of a:b " << c.putdata() ;
    return 1;
}
```

Parameterized Constructors: It is possible to pass arguments to constructors. Typically, these arguments help initialize an object when it is created. To create a parameterized constructor, simply add parameters to it the way you would to any other function. When you define the constructor's body, use the parameters to initialize the object

```
class construct
{
    int a, b;
public:
    // Default Constructor
    construct() { }
    //parameterized constructor
    construct(int x,int y)
    {
        a=x;
        b=y;
    }
}
```

```

    }

    void putdata() { cout<<a<<b; }
};

int main()
{
    // when the object is created
    construct c(5,10); //parameterized constructor
    cout << "value of a:b " << c.putdata() ;
    return 1;
}

```

Uses of Parameterized constructor:

1. It is used to initialize the various data elements of different objects with different values when they are created.
2. It is used to overload constructors.

Copy Constructor: A copy constructor is a member function which initializes an object using another object of the same class.

```

class Point
{
private:
    int x, y;
public:
    Point(int x1, int y1) { x = x1; y = y1; }

    // Copy constructor
    Point( Point &p2)
    {
        x = p2.x;
        y = p2.y;
    }
    void putdata() { cout<<x<<y; }

};

int main()
{
    Point p1(10, 15); // Parameterized constructor is called here
    Point p2 = p1; // Copy constructor is called here
    P1.putdata();
    P2.putdata();

    return 0;
}

```

}

What is destructor?

Destructor is a member function which destructs or deletes an object.

When is destructor called?

A destructor function is called automatically when the object goes out of scope:

- (1) the function ends
- (2) the program ends
- (3) a block containing local variables ends
- (4) a delete operator is called

How destructors are different from a normal member function?

Destructors have same name as the class preceded by a tilde (~)